

Kiến Trúc Kỹ Thuật, Phương Pháp Xử Lý & Quy Trình Huấn Luyện AI Chuyên Sâu

Dự án: **TrafficVision** – Hệ thống nhận dạng biển báo giao thông Việt Nam bằng AI

Tác giả: Nhóm nghiên cứu & phát triển TrafficVision

Kiến trúc: YOLO11 (Medium Production & Nano Candidate) + ONNX Runtime

Chuẩn phân loại: 82 Lớp QCVN 41:2019/BGTVT (10.129 ảnh)

Cập nhật: Tháng 10/2026 (Hoàn thành nghiệm thu thực nghiệm)

1. Kiến trúc Mô hình YOLO11 Nano & Nguyên lý Nhận dạng

Hệ thống lựa chọn **YOLO11 Nano (yolo11n)** do Ultralytics phát hành làm kiến trúc cơ bản nhờ tỷ lệ tối ưu giữa độ chính xác nhận dạng và độ trễ suy luận trên phần cứng CPU không có GPU rời.

[Ảnh đầu vào (Input 3x640x640)] | ▼ | 1. BACKBONE: Trích xuất đặc trưng đa quy mô | | - C3k2 Blocks (Cross Stage Partial với kernel tinh chỉnh) | | - SPPF (Spatial Pyramid Pooling - Fast) | | - C2PSA Block: Cơ chế Attention tự chú ý không gian | | (P3: 80x80, P4: 40x40, P5: 20x20) ▼ | 2. NECK: Dung hợp đa tỷ lệ PAN-FPN (Path Aggregation) | | - Ghép nối thông tin ngữ nghĩa sâu với vị trí nông | | - Tăng cường khả năng nhận diện biển báo kích thước nhỏ | | ▼ | 3. HEAD: Decoupled Head (Phân tách độc lập) | | - Nhánh 1: Phân loại lớp biển báo (Classification) | | - Nhánh 2: Hồi quy tọa độ hộp bao (Bounding Box Reg) |

Tại sao chọn YOLO11 Nano?

- Kích thước nhỏ gọn:** Số tham số ~2.6 triệu, trọng số FP32 ONNX chỉ khoảng **10.2 MB**.
- Khối C2PSA (Cross Stage Partial with Pointwise Spatial Attention):** Tăng cường khả năng tập trung vào các chi tiết biển báo nhỏ ở xa (Small Objects), vốn là điểm yếu của các kiến trúc YOLO thế hệ cũ.
- Decoupled Head:** Tách biệt việc dự đoán nhãn loại biển và định vị tọa độ hộp bao, giảm thiểu xung đột gradient khi huấn luyện.

2. Kỹ thuật Tiền xử lý & Chuẩn hóa Ảnh (Image Processing)

Để đảm bảo suy luận nhất quán và chính xác, mọi hình ảnh và khung hình video đều đi qua quy trình tiền xử lý toán học chuẩn hóa:

2.1. Thuật toán Letterbox (Bảo toàn tỷ lệ khung hình)

Khi ảnh đầu vào có tỷ lệ khác với tỷ lệ vuông 1:1 của mô hình (ví dụ ảnh 1920x1080 từ camera hành trình), thuật toán Letterbox sẽ:

- Tính toán tỷ lệ co giãn tối đa mà không làm biến dạng hình học: $gain = \min(640 / width, 640 / height)$.
- Resize ảnh theo tỷ lệ $gain$.
- Chèn thêm đệm màu xám (Pixel value = 114) đều vào hai bên hoặc trên/dưới để đạt đúng kích thước 640×640 .

$$\text{Tọa độ thực} = (\text{Tọa độ chuẩn hóa trên mô hình} - \text{Lượng padding}) / gain$$

2.2. Chuẩn hóa không gian màu & Tensor

- Chuyển đổi không gian màu từ BGR (OpenCV) sang **RGB**.
- Chuẩn hóa giá trị pixel từ dải số nguyên $[0, 255]$ sang số thực $[0.0, 1.0]$ bằng phép chia 255.0 .
- Hoán vị chiều dữ liệu (Permute) từ dạng HWC (Height x Width x Channel) sang **CHW**: $(1, 3, 640, 640)$.

3. Hàm Mất Mát (Loss Functions) & Chiến lược Tối ưu hóa

Quá trình huấn luyện sử dụng hàm tổn thất tổng hợp gồm 3 thành phần chính:

$$\text{Tổng Loss} = \lambda_{\text{box}} \cdot L_{\text{CIoU}} + \lambda_{\text{cls}} \cdot L_{\text{BCE}} + \lambda_{\text{dfl}} \cdot L_{\text{DFL}}$$

- CIoU Loss (Complete Intersection over Union):** Đo đặc độ tương đồng của hộp bao dự đoán và nhãn thật dựa trên: diện tích chồng lấn (IoU), khoảng cách giữa 2 tâm hộp và tỷ lệ cạnh dài/ngắn. Khắc phục hoàn toàn nhược điểm của hàm mất mát khoảng cách L1/L2 truyền thống.
- BCE Loss (Binary Cross-Entropy):** Tính toán tổn thất phân loại độc lập cho 82 lớp biển báo, hỗ trợ đa nhãn hiệu quả.
- DFL Loss (Distribution Focal Loss):** Hồi quy tọa độ hộp bao dưới dạng phân phối xác suất liên tục thay vì một điểm số cứng nhắc, giúp mô hình bắt viền biển báo sắc nét ngay cả khi bị che khuất một phần.

Chiến lược tối ưu hóa Hyperparameters:

Siêu tham số	Giá trị cấu hình	Ý nghĩa kỹ thuật
Optimizer	<code>AdamW</code> hoặc <code>SGD with Momentum</code>	Cân bằng tốc độ hội tụ và khả năng khái quát hóa dữ liệu.
Learning Rate ban đầu (lr0)	<code>0.01</code>	Tốc độ học ban đầu.
LR Scheduler	<code>Cosine Annealing</code>	Hạ dần learning rate theo đường cong cosin, giúp mô hình ổn định tại cực tiểu.
Mixed Precision (AMP)	<code>Enabled</code>	Tự động sử dụng Float16 trong quá trình tính toán để tăng tốc và tiết kiệm 50% VRAM.
Patience	<code>10 epochs</code>	Early stopping: Dừng huấn luyện nếu mAP50 trên tập Validation không tăng sau 10 epoch liên tiếp nhằm chống Overfitting.

4. Quy trình MLOps 4 Bước Khép Kín trong TrafficVision

Triết lý thiết kế: Mọi thử nghiệm đều phải có tính lặp lại (Reproducibility), có kiểm soát rủi ro và không được can thiệp vào môi trường phục vụ trực tuyến khi chưa được kiểm định đạt chuẩn.

Bước 1: Quét Dữ Liệu & Tiếp Nhận (Dataset Ingestion) & Dataset Repair

Hệ thống quét dữ liệu từ `artifacts/staging/` với 3 tập: `train/` (8.098 ảnh), `val/` (1.015 ảnh), `test/` (1.016 ảnh). Tổng quy mô: **10.129 ảnh** với **19.700 bounding boxes**.

Module Dataset Repair (`repair.py` / `scripts/repair_dataset.py`): Tự động sửa chữa các lỗi tọa độ làm tròn số (ví dụ tọa độ biên vượt quá 1.0), cắt gọt (clamp) chuẩn xác về miền `[0.0, 1.0]` và loại bỏ các hộp bao diện tích 0 trước khi nạp vào Quality Gate.

Bước 2: Cổng Kiểm Định Quality Gate 5 Tiêu Chí & Snapshot Bất Biến

Chặn đứng 5 loại lỗi nguy hiểm trước khi tốn tài nguyên huấn luyện:

- `CORRUPT_IMAGE` : Loại bỏ ảnh 0-byte, lỗi định dạng tệp hoặc ảnh không giải mã được.
- `MALFORMED_YOLO_LINE` : Kiểm tra cú pháp dòng nhãn (bắt buộc đúng 5 cột số: `class_id center_x center_y width height`).

- `CLASS_ID_OUT_OF_RANGE` : Đảm bảo class ID chỉ nằm trong dải [0, 81] tương ứng với danh mục QCVN 41:2019.
- `INVALID_COORDINATES` : Bắt buộc tọa độ tâm và kích thước nằm trong khoảng `(0, 1]`.
- `DATA_LEAKAGE` : Tính mã băm SHA-256 từng tệp ảnh. Nếu phát hiện cùng một ảnh xuất hiện ở cả tập Train và Val/Test, hệ thống sẽ chặn ngay lập tức để tránh hiện tượng học vẹt.

Khi vượt qua cổng kiểm định, hệ thống tạo **Snapshot bất biến** chính thức `snapshot_20260930_165016` kèm `data.yaml` và xuất báo cáo `eda_report.json`.

Bước 3: Huấn Luyện Nền Ngầm (Background Subprocess Execution) & Safe Resume

Thay vì chạy blocking làm treo giao diện Streamlit, tiến trình huấn luyện được ủy quyền cho một tiến trình hệ thống con độc lập qua module `TrainingManager`:

- **Phát hiện phần cứng thông minh (`hardware.py`)**: Tự động nhận diện GPU CUDA, MPS hoặc CPU. Tự động cấu hình `workers=0` trên Windows nhằm tránh lỗi rò rỉ RAM và xung đột đa tiến trình của PyTorch DataLoader.
- **Tính năng Safe Resume**: Cho phép tiếp tục chạy các epoch còn lại từ checkpoint `last.pt` khi phiên huấn luyện bị ngắt giữa chừng, bảo toàn 100% thời gian và tài nguyên đã bỏ ra.
- **Cập nhật tiến độ tức thì**: Tích hợp kỹ thuật `@st.fragment` tự động kiểm tra và reload thanh tiến trình, biểu đồ loss/mAP và nhật ký mỗi 3 giây.
- Ghi nhật ký thời gian thực vào `artifacts/runs/<run_id>/train.log` và trạng thái vào `state.json`.

Bước 4: Đóng Gói Ứng Viên (Candidate Packaging) & Thăng Cấp (Promotion)

Sau khi kết thúc huấn luyện, mô hình phải vượt qua 3 bài kiểm tra chất lượng tự động:

1. **Đánh giá độc lập trên tập Test (1.016 ảnh)**: Đo đặc Precision, Recall, mAP50, mAP50-95.
 - **YOLO11m (Production hiện hành)**: mAP50 đạt **98.03%**, Precision 96.16%, Recall 96.28%, F1 96.22%.
 - **YOLO11n (Candidate siêu nhẹ)**: mAP50 đạt **92.47%**, Latency 68.26 ms (~15 FPS).
2. **Xuất ONNX & Kiểm tra Parity**: Xuất sang ONNX đồ thị tĩnh `[1, 3, 640, 640]`. Chạy cùng một tensor đầu vào trên cả PyTorch và ONNX Runtime CPU, sau đó tính sai số tuyệt đối cực đại:

```
max_abs_diff = np.max(np.abs(torch_output - onnx_output))
assert max_abs_diff <= 1e-3, "Parity check failed!"
```

(Thực tế đo được trên YOLO11m: `0.000854 ≤ 1e-3` - Đạt chuẩn tuyệt đối).

3. **Benchmark CPU**: Xác định độ trễ và FPS trung bình.

Tính năng Finalize Checkpoint: Cho phép đóng gói Candidate từ checkpoint hoàn tất mà không cần huấn luyện lại.

Sau khi xác thực đầy đủ, mô hình YOLO11m đã được **thăng cấp lên Production** an toàn với cơ chế sao lưu tự động.

5. Kỹ Thuật Tối Ưu Hóa Suy Luận ONNX trên CPU

Nhằm đáp ứng yêu cầu chạy mượt trên máy tính văn phòng hoặc máy tính nhúng, các kỹ thuật sau được tích hợp trong `OnnxPredictor` :

- **Đồ thị tĩnh (Static Shape):** Khóa cứng kích thước input `1x3x640x640` giúp ONNX Runtime cấp phát trước bộ nhớ (pre-allocate memory), tránh hiện tượng phân mảnh RAM.
- **Graph Optimization Level:** Bật mức `ORT_ENABLE_ALL`, thực hiện gộp các phép tính liền kề (Constant Folding, Node Fusion như Conv + BatchNorm + SiLU thành một toán tử duy nhất).
- **Đa luồng CPU (Inter & Intra Op):** Thiết lập số luồng tính toán tương ứng với số nhân vật lý của CPU thông qua OpenMP/AVX2.
- **Hậu xử lý NMS tối ưu (Fast Non-Maximum Suppression):** Lọc nhanh các hộp bao có confidence < ngưỡng cài đặt, sau đó chỉ chạy IoU trên tập ứng viên còn lại, giảm độ phức tạp thuật toán từ $O(N^2)$ xuống $O(k \log k)$.

6. Cơ Chế An Toàn Vận Hành: Safe Atomic Swap & Rollback

Một điểm đột phá trong thiết kế của TrafficVision là sự đảm bảo an toàn tuyệt đối khi triển khai mô hình:

Người quản trị có thể kích hoạt hoàn tác bất kỳ lúc nào chỉ bằng lệnh dòng lệnh `python scripts/promote_model.py --rollback` hoặc bấm nút trên giao diện Web.